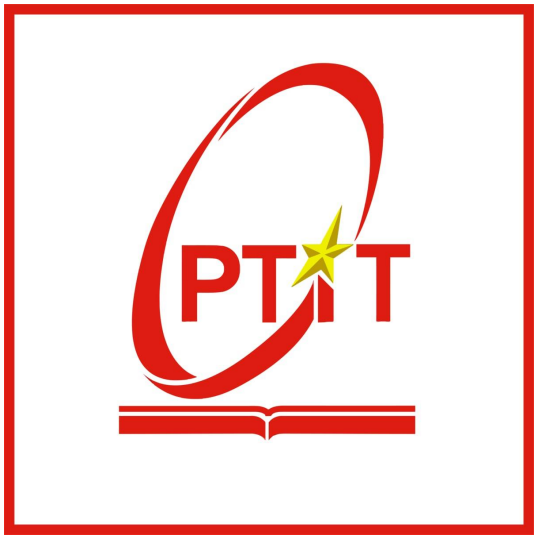
POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS



GROUP PROJECT REPORT - GROUP 6

F1 CHAMPIONSHIP MANAGEMENT SYSTEM

Supervisor: Assoc. Prof. Dr. Nguyen Trong Khanh

Team members: Nguyen Van Hieu - B23DCCE033
Pham Nhat Minh - B23DCCE066
Le Minh Duc - B23DCVT095

Hanoi, April 2026

Contents

List of Figures

List of Tables

Listings

Database Terminology Glossary

Term	Definition
ACID	A set of four properties (Atomicity, Consistency, Isolation, Durability) that guarantee the reliability of database transactions.
MVCC	(Multi-Version Concurrency Control) A mechanism that supports concurrent access by allowing multiple versions of data to exist simultaneously.
DBA	(Database Administrator) The administrator responsible for operating, securing, and optimizing the database system.
InnoDB engine	The default storage engine for MySQL that supports ACID transactions, row-level locking, and crash recovery features.
B+ Tree	A self-balancing tree data structure that optimizes search and block-based data retrieval in DBMS.
Buffer pool	A RAM memory area used to cache data and indexes, minimizing direct disk read/write operations.
Redo log	A log that records data changes to restore system state in the event of an unexpected failure.
Undo log	A log that records the previous state of data to support rollback operations and the MVCC mechanism.
Trigger	A SQL code object that automatically executes when an INSERT, UPDATE, or DELETE event occurs on a table.
Transaction	A logical unit of work consisting of a sequence of SQL operations, ensuring either complete execution or no execution at all.
Stored Procedure	A set of precompiled SQL statements stored on the server to execute recurring business operations.
Views	A virtual table that presents data from one or more tables through a predefined query statement.
Index	A data structure that speeds up record lookup, similar to the index of a book.
JWT Authentication	(JSON Web Token) A secure authentication method between system components using encoded tokens.
CTE	(Common Table Expression) A temporary result set that makes complex SQL statements clearer and easier to manage.

Chapter 1

Problem Introduction

1.1 Context

Formula 1 (F1) is the world's premier open-wheel racing championship. With dozens of teams, over 20 professional drivers, and a race calendar spanning nearly an entire year, managing the data lifecycle of this global sport demands precision and high data integrity. Each season comprises multiple races (Grand Prix) held across different countries.

The F1 Championship Management System is a comprehensive software platform designed to manage data across multiple seasons. Before having a centralized DBMS, managing driver contracts, team assignments, lap completion times, and scoring was highly error-prone. This system leverages the Relational Database Management System (RDBMS) model to provide a centralized, consistent, and highly available data repository.

1.2 System Objectives

The system is built to automate the core operations of an F1 season, specifically:

- Manage information about teams, drivers, and their active contracts.
- Schedule races and manage Grand Prix events across seasons.
- Register drivers for specific races based on strict business rules.
- Record race results in real-time and automatically calculate points according to the international FIA standard.
- Provide accurate and continuously updated driver and constructor standings.

1.3 Functional Requirements (FR)

The system must fulfill the following functional requirements:

FR-01 Manage F1 season information (Create, Update, Read).

FR-02 Manage the list of teams and drivers, including contract history.

FR-03 Allow driver registration for each race, with a constraint of a maximum of 2 drivers per team per race.

FR-04 Allow entry of race results (finish time, laps completed, status).

FR-05 Automatically calculate points according to the F1 standard (P1=25, P2=18, P3=15, etc.) immediately after saving results.

FR-06 Provide driver and constructor standings, with filtering support by race history.

FR-07 Support race result adjustments after saving, with automatic score recalculation.

1.4 Core Business Rules

The database logic is designed to strictly enforce the following rules:

1. **Driver limit:** A team is not allowed to register more than two drivers in the same race.
2. **Scoring system:** Points are calculated based on time ranking. Only drivers who finish the race (`status = 'Finished'`) are eligible for points.
3. **Time validity:** The finish time (`end_time`) must be greater than the race start time.
4. **Contract management:** When a driver transfers to another team, the old contract is set to `is_active = 0`, and a new contract is created with `is_active = 1`.
5. **Constructor points:** The total points of a team at a race equals the sum of points of all drivers belonging to that team at the race.

Chapter 2

Configuration, Tools, and Platform

During the development of the F1 Championship Management System, the selection of hardware, platform technologies, and software architecture plays a decisive role in the system's performance, data integrity, and scalability. Below is a detailed analysis of the environment and tools used by the team.

2.1 Hardware Environment and Deployment Configuration

The system was developed, tested, and load-tested on a personal computer serving as a local server. The hardware configuration not only supports running the source code but must also meet the demanding I/O requirements of the DBMS.

Table 2.1: Hardware configuration of the development environment

Component	Details and Rationale
Operating System	Windows 11 Pro 64-bit / Linux Ubuntu 22.04 LTS. Provides a stable environment for running MySQL and Node.js background services.
Processor (CPU)	Intel Core i5 12th Gen or equivalent. Ensures smooth processing of complex queries containing Window Functions and Subqueries.
Memory (RAM)	8 GB - 16 GB DDR4. **Significance for DBMS:** Provides sufficient capacity for the InnoDB Buffer Pool, enabling data and index caching in RAM, minimizing disk I/O.
Storage	256 GB SSD (NVMe preferred). **Significance for DBMS:** The high random write speed of SSDs is crucial for fast Redo Log and Undo Log writes, ensuring Transaction Durability.

2.2 Technologies Used

The application uses a modern technology stack, clearly separated via the 3-Tier Architecture model.

Table 2.2: Technology and platform overview

Tier	Technology	Version	Role
Database	MySQL / MariaDB	8.0 / 10.6+	Relational DBMS
Backend	Node.js	18.x LTS	JavaScript runtime environment
Backend	Express.js	4.x	RESTful API framework
DB Driver	mysql2/promise	3.x	Async Node.js-MySQL connector
Frontend	React.js (Vite)	18.x	SPA UI rendering library
Frontend	TailwindCSS	3.x	Utility-first CSS framework
DB Admin	MySQL Workbench	8.x	DB design and query tool

2.3 Architecture Rationale

The system is designed following the **3-Tier Architecture**:

1. **Presentation Tier (React.js):** Provides a smooth Single Page Application (SPA) experience. Vite is used to leverage extremely fast build speeds (HMR).

2. **Application Tier (Node.js/Express.js):** Acts as middleware for processing logic. Using JavaScript for both Frontend and Backend reduces context-switching costs. The `mysql2` library was chosen for its Promise API support, enabling safe `async/await` usage and efficient connection pool management.

3. **Data Tier (MySQL/InnoDB):** MySQL was chosen for its excellent support for the relational model, ACID transactions, Stored Procedures, and Triggers. The InnoDB engine plays a critical role with its row-level locking mechanism, preventing bottlenecks when multiple threads update race results simultaneously.

Chapter 3

Installation & Deployment

This chapter describes in detail the database design process, SQL scripts (DDL, Triggers, Procedures, Views), and deployment steps.

3.1 Database Design and Normalization

The database schema was rigorously designed to achieve **BCNF (Boyce-Codd Normal Form)**, ensuring data integrity and completely eliminating redundancy.

3.1.1 Relational Schema

- `CHAMPIONSHIPS(champ_code, name, description)`
- `TEAMS(team_code, name, brand, owner, description)`
- `DRIVERS(driver_code, name, date_of_birth, nationality)`
- `CONTRACTS(contract_id, driver_code*, team_code*, is_active)`
- `RACES(race_code, name, num_laps, start_time, champ_code*)`
- `RACE_ENTRIES(entry_id, race_code*, contract_id*)`
- `RESULTS(entry_id*, end_time, laps_completed, status, points)`

(denotes Foreign Key)*

Detailed Explanation:

The system consists of 7 main entities (tables), designed to comprehensively manage the lifecycle of an F1 racing season.

- **CHAMPIONSHIPS (Seasons):** Stores information about racing seasons.
  - *Attributes:* `champ_code` (VARCHAR(10) - Season code - PK), `name` (VARCHAR(255) - Season name), `description` (TEXT - Description).
- **TEAMS (Racing Teams):** Manages information about participating teams.
  - *Attributes:* `team_code` (VARCHAR(10) - Team code - PK), `name` (VARCHAR(100) - Team name), `brand` (VARCHAR(100) - Car brand), `owner` (VARCHAR(100) - Owner), `description` (TEXT - Description).
- **DRIVERS (Drivers):** Manages driver profile information.
  - *Attributes:* `driver_code` (VARCHAR(10) - Driver code - PK), `name` (VARCHAR(100) - Name), `date_of_birth` (DATE - Date of birth),

`nationality` (VARCHAR(50) - Nationality), `biography` (TEXT - Biography).

- **CONTRACTS (Contracts):** Junction table storing the signing history between Drivers and Teams.
  - *Attributes:* `contract_id` (INT - Contract ID - PK), `driver_code` (VARCHAR(10) - Driver code - FK), `team_code` (VARCHAR(10) - Team code - FK), `is_active` (TINYINT - Active status).
- **RACES (Races):** Information about each race in a season.
  - *Attributes:* `race_code` (VARCHAR(10) - Race code - PK), `name` (VARCHAR(100) - Race name), `num_laps` (INT - Number of laps), `location` (VARCHAR(255) - Location), `start_time` (DATETIME(3) - Start time), `champ_code` (VARCHAR(10) - Season code - FK), `description` (TEXT - Description).
- **RACE_ENTRIES (Race Entries):** Junction table recording driver registration (via contract) for a specific race.
  - *Attributes:* `entry_id` (INT - Entry ID - PK), `race_code` (VARCHAR(10) - Race code - FK), `contract_id` (INT - Contract ID - FK).
- **RESULTS (Race Results):** Records driver performance after a race.
  - *Attributes:* `entry_id` (INT - Entry ID - PK/FK), `end_time` (DATETIME(3) - Finish time), `laps_completed` (INT - Laps completed), `status` (ENUM - Status Finished/DNF/Accident), `points` (INT - Points).

Based on the foreign key structure, the entities in the system have the following relationships:

1. **Championship - Race (1:N):** A championship (`CHAMPIONSHIPS`) includes multiple races (`RACES`).
2. **Team - Contract (1:N):** A team (`TEAMS`) can sign multiple contracts (`CONTRACTS`) with different drivers.
3. **Driver - Contract (1:N):** A driver (`DRIVERS`) can have multiple contracts (`CONTRACTS`) with different teams throughout their career.
4. **Race - Entry (1:N):** Each race (`RACES`) has multiple race entries (`RACE_ENTRIES`).
5. **Contract - Entry (1:N):** A contract (`CONTRACTS` - representing 1 driver of 1 team) can be used to register for multiple races (`RACE_ENTRIES`).
6. **Entry - Result (1:1):** Each race entry (`RACE_ENTRIES`) has exactly one corresponding result record (`RESULTS`).

3.1.2 Normalization Assessment

- **First Normal Form (1NF):** All attributes are atomic. There are no repeating groups. For example: a team's driver list is managed through multiple records in `CONTRACTS` rather than using a comma-separated text string.

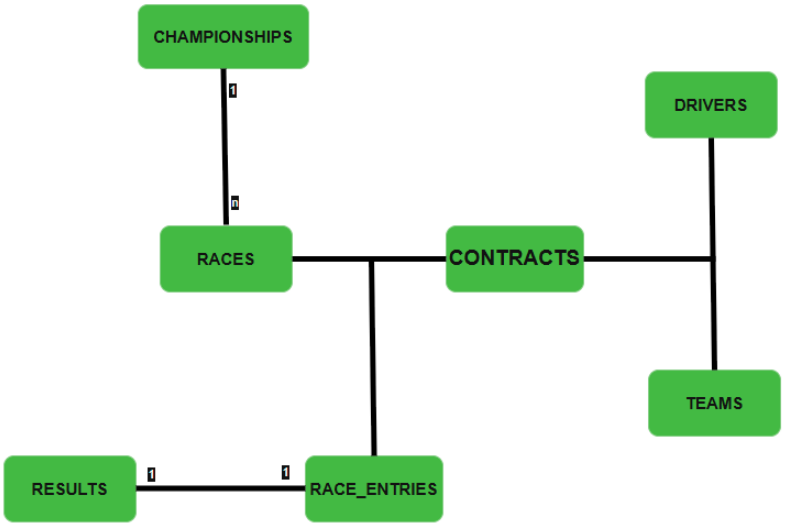


Figure 3.1: Diagram illustrating the 3-tier architecture of the system

- **Second Normal Form (2NF):** All tables satisfy 1NF and have no partial dependencies. The `RACE_ENTRIES` table uses a surrogate key (`entry_id`) instead of a composite key, completely eliminating the possibility of 2NF violations.
- **Third Normal Form (3NF) & BCNF:** The schema eliminates all transitive dependencies. For instance, `team_name` does not appear in `RACE_ENTRIES`. Every determinant in the system is a superkey.

3.2 Data Definition Language (DDL) Script

The following SQL code initializes the database, table structures, and establishes foreign keys.

```sql
DROP DATABASE IF EXISTS F1_Championship_Management;
CREATE DATABASE F1_Championship_Management;
USE F1_Championship_Management;

SET SQL_SAFE_UPDATES = 0;

CREATE TABLE CHAMPIONSHIPS (
    champ_code  VARCHAR(10)  PRIMARY KEY,
    name        VARCHAR(255) NOT NULL,
    description TEXT
);

CREATE TABLE TEAMS (
    team_code   VARCHAR(10)  PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    brand       VARCHAR(100),
    owner       VARCHAR(100),
    description TEXT
);

```

```sql
CREATE TABLE DRIVERS (
    driver_code   VARCHAR(10)  PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    date_of_birth DATE,
    nationality   VARCHAR(50),
    biography     TEXT
);

CREATE TABLE CONTRACTS (
    contract_id INT          AUTO_INCREMENT PRIMARY KEY,
    driver_code VARCHAR(10),
    team_code   VARCHAR(10),
    is_active   TINYINT      DEFAULT 1,
    FOREIGN KEY (driver_code) REFERENCES DRIVERS(driver_code),
    FOREIGN KEY (team_code)   REFERENCES TEAMS(team_code)
);

CREATE TABLE RACES (
    race_code   VARCHAR(10)  PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    num_laps    INT          NOT NULL,
    location    VARCHAR(255),
    start_time  DATETIME(3),
    champ_code  VARCHAR(10),
    description TEXT,
    FOREIGN KEY (champ_code) REFERENCES CHAMPIONSHIPS(
       champ_code)
);

CREATE TABLE RACE_ENTRIES (
    entry_id    INT AUTO_INCREMENT PRIMARY KEY,
    race_code   VARCHAR(10),
    contract_id INT,
    FOREIGN KEY (race_code)   REFERENCES RACES(race_code),
    FOREIGN KEY (contract_id) REFERENCES CONTRACTS(contract_id)
       ,
    UNIQUE(race_code, contract_id)
);

CREATE TABLE RESULTS (
    entry_id       INT PRIMARY KEY,
    end_time       DATETIME(3),
    laps_completed INT,
    status         ENUM('Finished', 'DNF', 'Accident') DEFAULT
       'Finished',
    points         INT DEFAULT 0,
    FOREIGN KEY (entry_id) REFERENCES RACE_ENTRIES(entry_id),
    CONSTRAINT chk_laps_completed CHECK (laps_completed >= 0),
    CONSTRAINT chk_points         CHECK (points >= 0)
);
```

Listing 3.1: DDL — Table creation and constraint setup

3.3 Database Triggers for Business Logic

Embedding business logic at the database layer ensures that rules cannot be bypassed, regardless of whether data is entered via the API or directly through a DB admin tool.

3.3.1 Limiting the Number of Drivers per Team

Purpose: Prevent the registration of a 3rd driver from the same team in the same race.

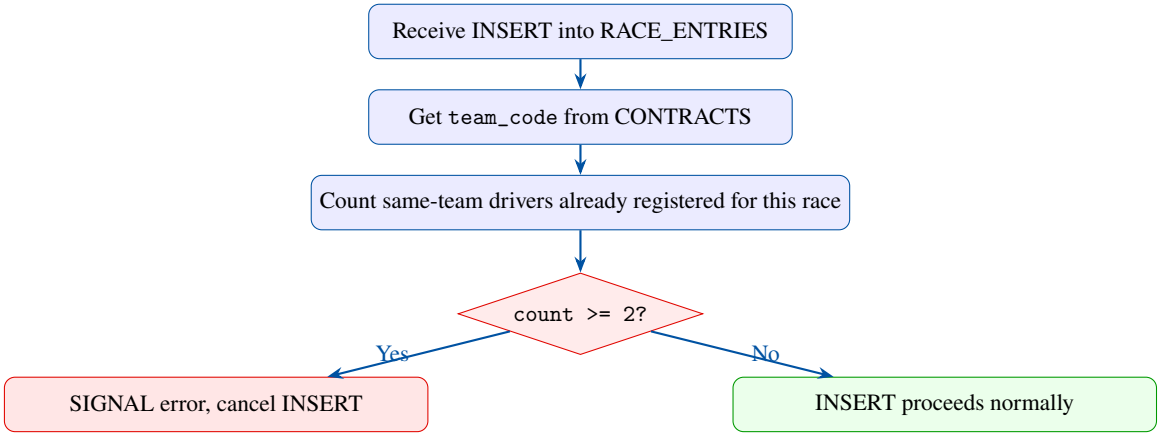


Figure 3.2: Processing flow of Trigger trg_limit_2_riders

This trigger ensures no team registers more than 2 drivers in the same race.

```
DELIMITER //
CREATE TRIGGER trg_limit_2_riders
BEFORE INSERT ON RACE_ENTRIES
FOR EACH ROW
BEGIN
    DECLARE v_team_id VARCHAR(10);
    DECLARE v_count INT;

    -- Lay ma doi cua tay dua sap dang ky
    SELECT team_code INTO v_team_id
    FROM CONTRACTS WHERE contract_id = NEW.contract_id;

    -- Dem so tay dua cua doi nay da dang ky trong chang
    SELECT COUNT(*) INTO v_count
    FROM RACE_ENTRIES re
    JOIN CONTRACTS c ON re.contract_id = c.contract_id
    WHERE re.race_code = NEW.race_code
      AND c.team_code = v_team_id;

    -- Huy giao dich neu vuot qua so luong cho phep
    IF v_count >= 2 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Each team can register a
           maximum of 2 drivers!';
    END IF;
END //
DELIMITER ;
```

Listing 3.2: Trigger — trg_limit_2_riders

3.3.2 Time Validity Check

Purpose: Ensure `end_time > start_time` of the race. Applied to both INSERT and UPDATE on the RESULTS table. This trigger ensures the finish time occurs after the race start time.

```
DELIMITER //
CREATE TRIGGER trg_check_time_insert
BEFORE INSERT ON RESULTS
FOR EACH ROW
BEGIN
    DECLARE v_start_time DATETIME(3);
    IF NEW.status = 'Finished' AND NEW.end_time IS NOT NULL
       THEN
        SELECT r.start_time INTO v_start_time
        FROM RACE_ENTRIES re
        JOIN RACES r ON re.race_code = r.race_code
        WHERE re.entry_id = NEW.entry_id;

        IF NEW.end_time <= v_start_time THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
              'Error: Finish time must be greater than start
                 time!';
        END IF;
    END IF;
END //
DELIMITER ;
```

Listing 3.3: Trigger — trg_check_time_insert

Rationale for database-level constraints: If validation is only performed at the Frontend or Backend, users can bypass it using Postman or MySQL Workbench. Triggers ensure constraints are enforced at **EVERY ACCESS POINT**.

3.4 Stored Procedure and Transaction

Purpose: Automatically calculate F1 points based on time ranking, wrapped in an ACID Transaction.

F1 point calculation requires ranking drivers by their finish times. Performing this dynamically via a View would degrade performance. Therefore, a Stored Procedure is called immediately after saving results. This process runs inside a Transaction to ensure Atomicity.

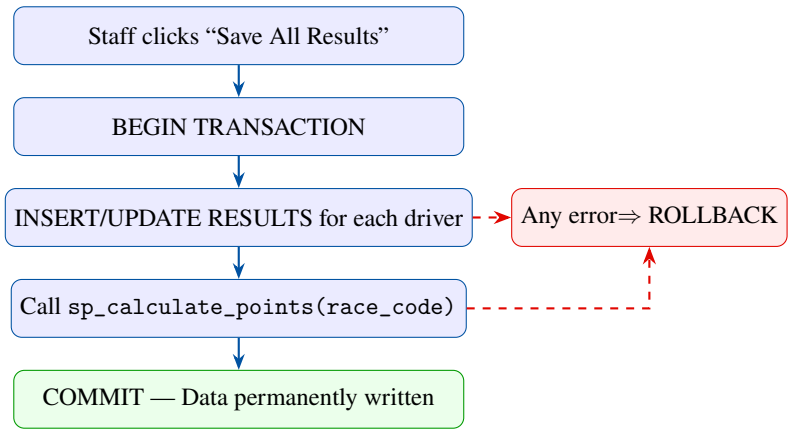


Figure 3.3: Transaction flow of Module 2 — Result Entry

```sql
DELIMITER //
CREATE PROCEDURE sp_calculate_points(IN p_race_code VARCHAR(10)
    )
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Buoc 1: Reset diem chang ve 0
    UPDATE RESULTS res
    JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
    SET res.points = 0
    WHERE re.race_code = p_race_code;

    -- Buoc 2: Xep hang thoi gian va cap nhat diem
    UPDATE RESULTS target
    JOIN (
        SELECT res.entry_id,
        RANK() OVER (
            ORDER BY TIMESTAMPDIFF(MICROSECOND, r.start_time,
               res.end_time) ASC
        ) AS pos
        FROM RESULTS res
        JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
        JOIN RACES r ON re.race_code = r.race_code
        WHERE re.race_code = p_race_code AND res.status = '
           Finished'
    ) rnk ON target.entry_id = rnk.entry_id
    SET target.points = CASE
        WHEN rnk.pos = 1  THEN 25  WHEN rnk.pos = 2  THEN 18
        WHEN rnk.pos = 3  THEN 15  WHEN rnk.pos = 4  THEN 12
        WHEN rnk.pos = 5  THEN 10  WHEN rnk.pos = 6  THEN 8
        WHEN rnk.pos = 7  THEN 6   WHEN rnk.pos = 8  THEN 4
        WHEN rnk.pos = 9  THEN 2   WHEN rnk.pos = 10 THEN 1
        ELSE 0
```

```
    END;

    COMMIT;
END //
DELIMITER ;
```

Listing 3.4: Stored Procedure sp_calculate_points — Automatic F1 point calculation

3.5 Views and Indexing

Views are used to encapsulate complex JOIN queries and aggregate functions, allowing the Application tier to use simple SELECT statements.

```
CREATE VIEW v_race_performance AS
SELECT re.race_code, c.driver_code, c.team_code,
       res.status, res.laps_completed, res.points,
       TIMESTAMPDIFF(MICROSECOND, r.start_time, res.end_time) /
           1000000
           AS finish_time_seconds,
       r.start_time AS race_start_time
FROM RESULTS res
JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
JOIN RACES r ON re.race_code = r.race_code
JOIN CONTRACTS c ON re.contract_id = c.contract_id;

CREATE VIEW v_driver_standings AS
SELECT d.driver_code, d.name, d.nationality, t.name AS
   team_name,
       SUM(v.points) AS total_score,
       SUM(CASE WHEN v.status = 'Finished'
                THEN v.finish_time_seconds ELSE 0 END) AS
                   total_season_time
FROM DRIVERS d
JOIN CONTRACTS c ON d.driver_code = c.driver_code AND c.
   is_active = 1
JOIN TEAMS t ON c.team_code = t.team_code
JOIN v_race_performance v ON v.driver_code = d.driver_code
                          AND v.team_code  = t.team_code
GROUP BY d.driver_code, d.name, d.nationality, t.name
ORDER BY total_score DESC, total_season_time ASC;
```

Listing 3.5: Views — Driver and Constructor Standings

To optimize queries scanning thousands of records, the following additional Indexes are created:

```
CREATE INDEX idx_race_code           ON RACE_ENTRIES(race_code)
   ;
CREATE INDEX idx_team_code_contracts ON CONTRACTS(team_code,
   driver_code);
CREATE INDEX idx_results_status_time ON RESULTS(status,
   end_time);
CREATE INDEX idx_races_start_time    ON RACES(start_time);
```

Listing 3.6: Performance optimization with Indexes

3.6 Deployment Guide

3.6.1 Step 1: Initialize the Database

Clone the project repository to your machine. Open MySQL terminal or MySQL Workbench, log in with an admin account and run the SQL file:

```
mysql -u root -p
# Nhap mat khau khi duoc yeu cau
SOURCE /path/to/project/database/schema.sql;
```

Verify with the command: `USE F1_Championship_Management; SHOW TABLES;`.

3.6.2 Step 2: Configure Backend

Navigate to the `backend` directory. Requires Node.js (v18+) installed.

```
cd backend
npm install
```

Create a `.env` file in the root of the `backend` directory with the following content:

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=F1_Championship_Management
PORT=3000
```

Start the Express server: `npm run dev`.

3.6.3 Step 3: Launch Frontend

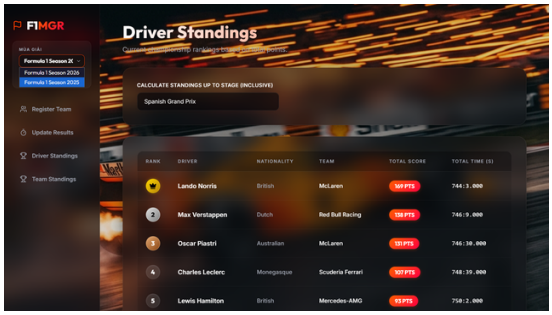
Open a new terminal and navigate to the `frontend` directory.

```
cd frontend
npm install
npm run dev
```

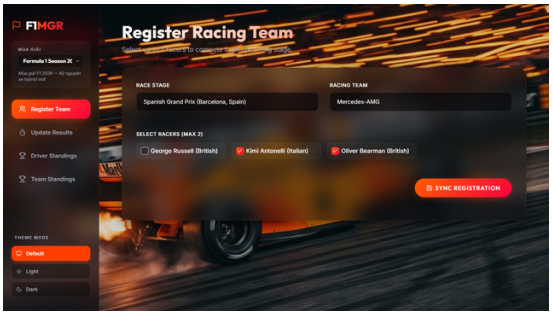
The application will run at `http://localhost:5173`. You can access the "Standings" module to verify the standings feature, or the "Update Results" module to test Transaction and Stored Procedure flows.

3.7 Results

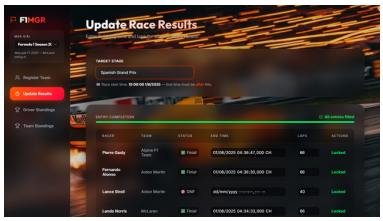
Full project source code: https://github.com/DucNet911/F1ChampionshipDBMS



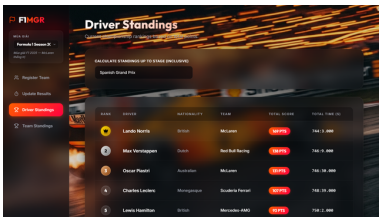
(a) Season management interface



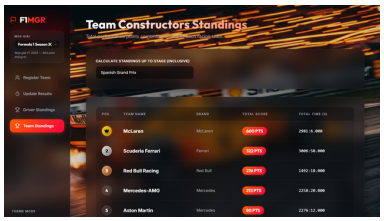
(b) Driver registration interface



(c) Result update interface



(d) Driver standings interface



(e) Constructor standings interface

Figure 3.4: F1 Championship Management System interface

Chapter 4

Conclusion

4.1 Summary of Achievements

The development and deployment of the F1 Championship Management System has demonstrated the practical application of advanced DBMS concepts. The completed system achieved the following:

- A database schema of 7 tables achieving BCNF, thoroughly resolving insertion, update, and deletion anomalies.
- 6 Foreign Keys established to ensure referential integrity.
- 3 Database Triggers strictly enforcing business rules (driver limits, time validity) completely independent of the application layer.
- Centralized automatic point calculation through 1 Stored Procedure utilizing the `RANK()` function within a safe ACID Transaction.
- Data abstraction through 3 Views, combined with 4 Indexes optimizing retrieval speed.

4.2 Lessons Learned

The project reinforced several important principles in database design:

1. **Database as the Single Source of Truth:** Embedding the 2-driver limit logic directly into a MySQL Trigger permanently protects the system from API client errors.
2. **Transaction Scope:** When simultaneously updating results for 20 drivers, a Transaction is mandatory. Without `START TRANSACTION` and `COMMIT/ROLLBACK`, a single mid-process error would corrupt the entire standings.
3. **Normalize First, Code Later:** Separating the `CONTRACTS` table instead of embedding `team_code` directly into `DRIVERS` prevented countless data synchronization bugs later on.

4.3 Future Development

Although fully functional, the system can be further extended:

- **Handling Race Conditions:** Currently, the Trigger counts drivers using `SELECT COUNT`. Under extremely high concurrent load, race conditions may occur. An upgrade to `SELECT ... FOR UPDATE` should be implemented to lock records during processing.
- **Detailed Contract History Tracking:** Upgrade the `CONTRACTS` table by adding `start_date` and `end_date` instead of using only the `is_active` flag, enabling full contract lifecycle retrieval.

- **System Security:** Integrate Role-Based Access Control (RBAC) with JSON Web Tokens (JWT) at the Node.js layer to restrict result writing permissions to organizers only.
- **Audit Logging:** Create an `AUDIT_LOG` table and set up `AFTER UPDATE` triggers to precisely record who modified sensitive result data and when.

References

[1] Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.

[2] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

[3] Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

[4] MariaDB Foundation. (2024). *MariaDB Server Documentation*. Retrieved from https://mariadb.com/kb/en/documentation/

[5] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.

[6] Fédération Internationale de l'Automobile (FIA). (2024). *Formula 1 Sporting Regulations*. Retrieved from https://www.fia.com/